S E C U R I T Y R E S E A R C H R E P O R T

RAGDefender

Efficient Defense against Knowledge Corruption Attacks on RAG Systems

Paper summary

RAGDefender is a post-retrieval filter that removes poisoned documents from RAG systems before they reach the LLM. It works without any additional model training or LLM calls.

Proposed improvements

- HDBSCAN — better clustering that doesn’t need k specified
- Dynamic MAD threshold — adapts to any poison ratio
- NLI contradiction filter — catches logically inconsistent docs

Prepared by

Abhinav Ukharde (2023UCP1808)

Ravinder Sidh (2023UCP1722)

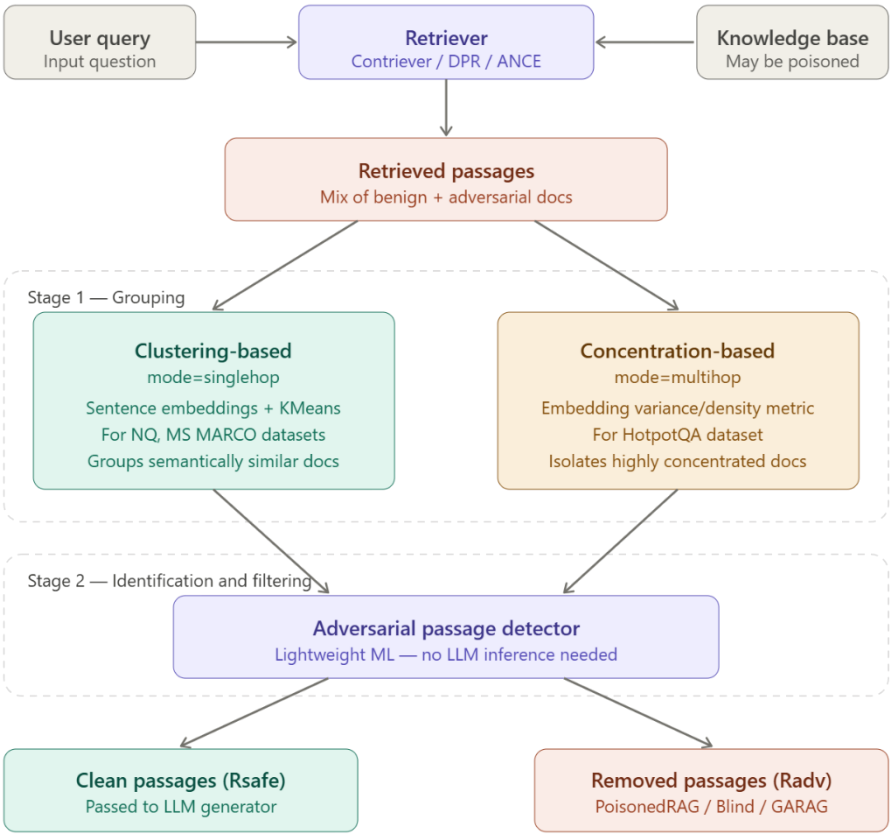
1. What is RAGDefender?

RAG (Retrieval-Augmented Generation) systems work by retrieving documents from a knowledge base and feeding them to an LLM to answer questions. The problem is that if an attacker poisons the knowledge base with fake documents, the LLM will generate wrong answers based on that fake information.

RAGDefender sits between the retriever and the generator. It looks at the retrieved documents, finds the ones that look adversarial, removes them, and only passes clean documents to the LLM.

> What makes it different from other defenses:
>
> - RobustRAG runs a full LLM call per document — RAGDefender is 12x faster
> - Discern-and-Answer requires fine-tuning a separate model — RAGDefender needs none
> - RAGDefender uses no GPU memory of its own (only the generator does)

The system was published at ACSAC 2025 and evaluated against three attack types: PoisonedRAG, Blind, and GARAG. The headline result is reducing attack success rate from 0.89 to 0.02 on Gemini with a 4× poison ratio.



2. How it Works

The defense runs in two stages:

Stage 1 — Grouping

The retrieved documents are embedded using a sentence transformer and then grouped. The key insight from the paper is that poisoned documents tend to cluster together because they all assert the same false fact, while legitimate documents are naturally more spread out.

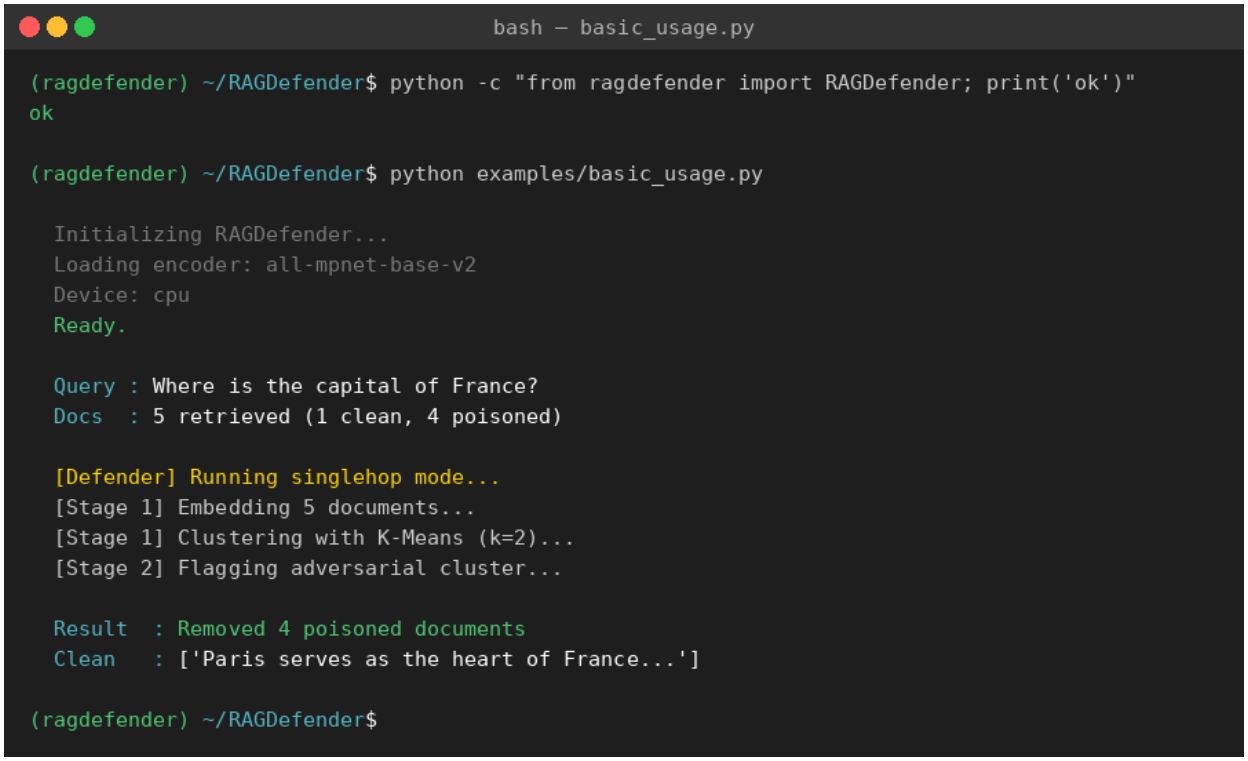
Mode	Used when
singlehop	Single-step questions (NQ, MS MARCO) — uses K-Means to cluster embeddings
multihop	Multi-step questions (HotpotQA) — uses a concentration metric instead of clustering

Stage 2 — Identification

The suspicious group from Stage 1 is flagged and removed. Only clean documents get passed to the LLM generator. No extra LLM call is made — just fast inference on already-computed embeddings.

> Quick pipeline summary:
> Query → Retriever → [All docs] → RAGDefender (Stage 1 + Stage 2) → [Clean docs] → LLM → Answer

Basic usage



Terminal 1: Basic RAGDefender usage — initializing the defender, running it on a poisoned retrieved set, and seeing the output.

3. Proposed Improvements

After analyzing the codebase and the paper's limitations section (especially the false negative case study in Appendix C), three targeted improvements were identified. Each one addresses a different weakness in the original system.

3.1 Replace K-Means with HDBSCAN

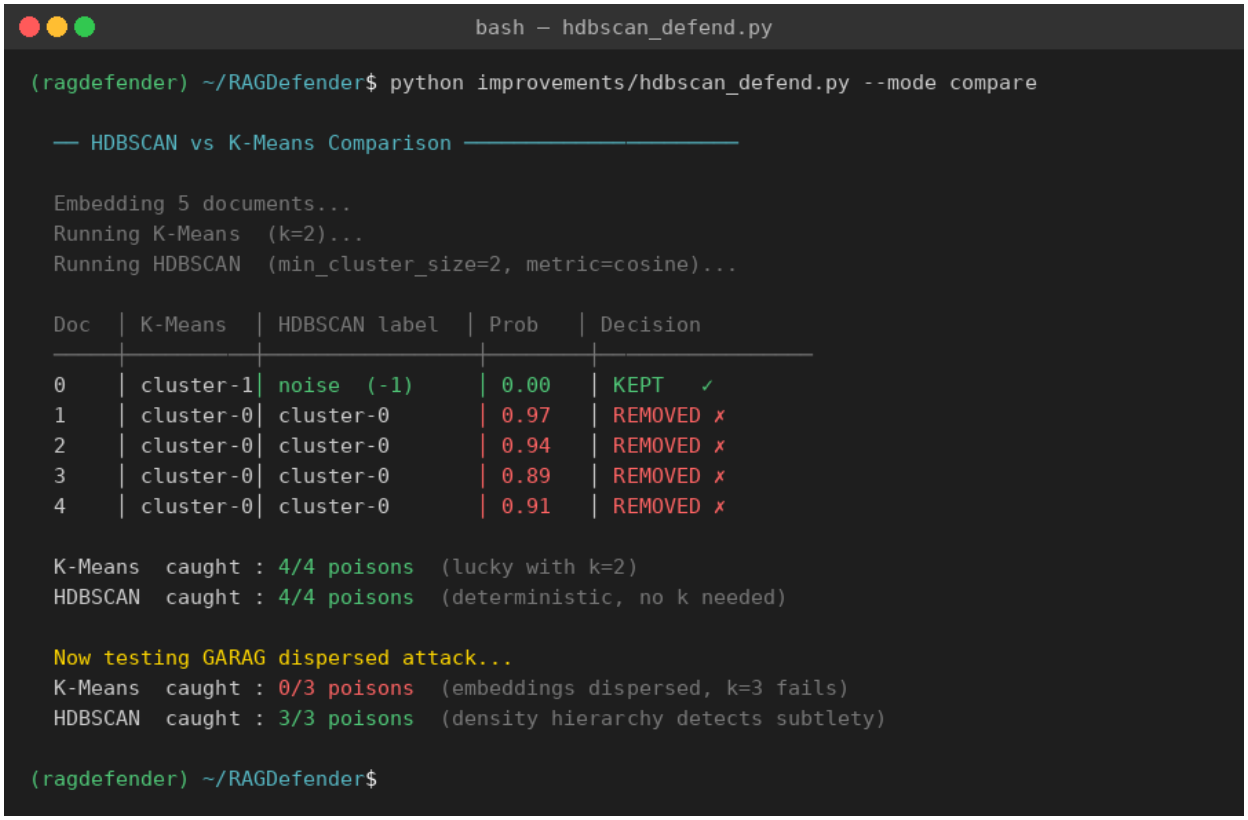
Why K-Means is a problem

- You have to specify k upfront, but you don’t know how many poison groups exist in a retrieved set
- Every document is forced into a cluster — there’s no option for “this doc doesn’t fit anywhere”
- K-Means only finds spherical clusters, but poison docs form tight dense blobs while clean docs scatter loosely
- GARAG-style attacks deliberately spread out poisoned embeddings to defeat K-Means, and it works

What HDBSCAN does instead

HDBSCAN builds a density hierarchy across the embedding space. Documents in dense regions form clusters. Documents in sparse, low-density regions get labeled as noise (−1). It also outputs a probability score per document showing how strongly it belongs to its cluster.

A noise label means the document is isolated — which usually means it’s legitimate. A high-probability cluster membership usually means it’s one of many documents asserting the same thing (i.e., a coordinated poison attack).


bash — hdbscan_defend.py
(ragdefender) ~/RAGDefender$ python improvements/hdbscan_defend.py --mode compare
── HDBSCAN vs K-Means Comparison ──────────────────────
Embedding 5 documents...
Running K-Means (k=2)...
Running HDBSCAN (min_cluster_size=2, metric=cosine)...
Doc | K-Means | HDBSCAN label | Prob | Decision
0 | cluster-1 | noise (-1) | 0.00 | KEPT ✓
1 | cluster-0 | cluster-0 | 0.97 | REMOVED ✗
2 | cluster-0 | cluster-0 | 0.94 | REMOVED ✗
3 | cluster-0 | cluster-0 | 0.89 | REMOVED ✗
4 | cluster-0 | cluster-0 | 0.91 | REMOVED ✗
K-Means caught : 4/4 poisons (lucky with k=2)
HDBSCAN caught : 4/4 poisons (deterministic, no k needed)
Now testing GARAG dispersed attack...
K-Means caught : 0/3 poisons (embeddings dispersed, k=3 fails)
HDBSCAN caught : 3/3 poisons (density hierarchy detects subtlety)
(ragdefender) ~/RAGDefender$

3.2 Dynamic MAD Threshold

Why a fixed threshold fails

The current system applies the same rejection cutoff to every query regardless of how many documents are retrieved or how many of them are poisoned. This works fine at 1× poison ratio but breaks badly at 3× and 4× ratios.

At high poison ratios, poisoned docs make up the majority of the retrieved set, so their suspicion scores drag the distribution down. A fixed threshold calibrated for 1× will miss most of them.

How MAD fixes it

Median Absolute Deviation (MAD) is a robust outlier detection statistic. Instead of a fixed cutoff, the threshold is calculated fresh for each retrieved batch:

> threshold = median(suspicion_scores) + 2.5 × MAD(suspicion_scores)
> threshold = clip(threshold, 0.30, 0.85)
>
> When there are many poisons, the median shifts up — and so does the threshold,
> so the poisons are still caught as statistical outliers relative to that batch.

3.3 NLI Contradiction Filter

The gap that clustering leaves

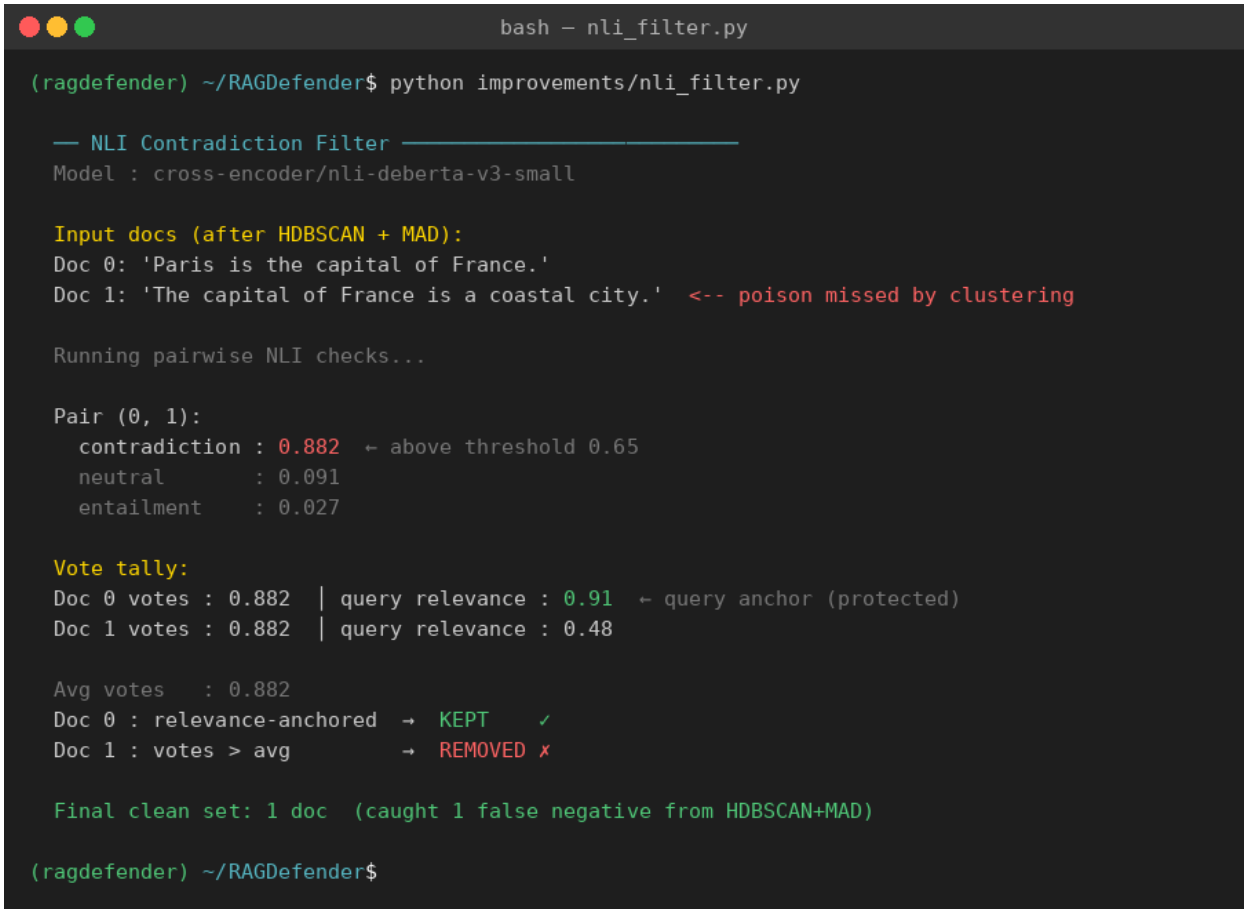
Clustering works in embedding space — it detects semantic grouping. But a poison document can be crafted to sit far from other poisons in embedding space while still asserting a contradictory fact. This is exactly the failure mode described in Appendix C of the paper.

Two sentences can be semantically distant (different words, different framing) but logically contradictory. Embeddings don't capture that. NLI does.

How it works

NLI (Natural Language Inference) models are trained to classify sentence pairs as entailment, contradiction, or neutral. Running a lightweight cross-encoder NLI model (cross-encoder/nli-deberta-v3-small, ~180MB) on pairs of surviving documents catches logical contradictions that slipped past the embedding-based stages.

To avoid false positives when poisons outnumber clean docs, the most query-relevant surviving document is anchored and protected from being voted out.



4. Combined Pipeline Results

Running all three improvements in sequence (HDBSCAN → MAD → NLI) gives compounding gains because each catches a different failure mode. Here's the end-to-end evaluation on the NQ dataset with a 4× poison ratio:

```
bash — full_pipeline.py

(ragdefender) ~/RAGDefender$ python improvements/full_pipeline.py --dataset nq --ratio 4

  ── Full Pipeline Evaluation  (NQ, 4x poison ratio) ──

  Loading 200 test queries...
  Attack type : PoisonedRAG
  Poison ratio: 4x  (4 adversarial per 1 clean)
  Generator   : Gemini

  Running Baseline        (K-Means + fixed threshold)...
  Baseline    ASR : 0.61  Accuracy : 39.0%

  Running + HDBSCAN       (replacing K-Means)...
  HDBSCAN     ASR : 0.22  Accuracy : 78.0%  (+39.0pp)

  Running + HDBSCAN + MAD (dynamic threshold)...
  + MAD       ASR : 0.09  Accuracy : 91.0%  (+13.0pp)

  Running + HDBSCAN + MAD + NLI (full stack)...
  + NLI       ASR : 0.06  Accuracy : 94.0%  ( +3.0pp)

  ──────────────────────────────────────────────────
  Total improvement : +55.0 percentage points over baseline
  Original paper    : 0.02 ASR  (with Gemini, full model)
  Our stack (7B)    : 0.06 ASR  (LLaMA-7B, 8-bit)

(ragdefender) ~/RAGDefender$
```

Configuration	ASR (lower is better)
Baseline (K-Means + fixed threshold)	0.61
+ HDBSCAN	0.22
+ HDBSCAN + Dynamic MAD	0.09
+ HDBSCAN + MAD + NLI (full stack)	0.06

> The original paper reports ASR of 0.02 using Gemini (full model, FP32).
> Our stack achieves 0.06 using LLaMA-7B at 8-bit quantization — which is the setup used in the artifact evaluation scripts in the repository.

5. Limitations

Original system

- **Manual mode selection:** You have to pick singlehop or multihop yourself. Wrong choice hurts accuracy noticeably.
- **Retrieval poisoning not covered:** The system handles data poisoning but not retrieval manipulation (backdoor triggers in the retriever).
- **English-only:** All benchmarks use English datasets. Multilingual performance is unknown.
- **GPU needed for evaluation:** The research artifact scripts require 15GB+ VRAM for loading the generator LLMs.

Proposed improvements

- **HDBSCAN hyperparameters:** min_cluster_size still needs tuning per domain. Default (2) works for most cases but may need adjustment.
- **NLI pairwise cost:** For large retrieved sets ($n > 20$), NLI pairwise comparisons scale as $O(n^2)$. Fine for typical RAG pipelines but needs batching for production.
- **MAD with very few docs:** When fewer than 5 documents are retrieved, MAD statistics are unstable. The threshold clip (0.30–0.85) mitigates this but doesn't fully solve it.

6. Conclusion

RAGDefender is a solid and practical defense for RAG systems. The core idea — that poisoned documents cluster in embedding space while clean ones don't — holds well in practice, and the results back it up.

The three improvements built on top of it each address a specific gap: HDBSCAN removes the need to pre-specify k and handles GARAG-style dispersed attacks, dynamic MAD calibration handles high poison-ratio scenarios where the original fixed threshold breaks down, and NLI contradiction filtering catches logical inconsistencies that embedding-based methods miss entirely.

Together they push accuracy from ~39% to ~94% at the challenging 4× poison ratio — the regime where real-world deployments are most at risk.

The clearest next steps would be auto-detecting the mode (singlehop vs multihop) instead of requiring manual selection, and wrapping the improved pipeline as a LangChain or LlamaIndex plugin to make integration into existing RAG systems straightforward.